

QUERYHOLIC

Comprehensive Frontend Engineering Architecture & Workflow
Production-Ready Implementation from 453 Stitch-Generated Screens

Framework: Next.js (App Router) + TypeScript	Styling: Tailwind CSS + shadcn/ui
State/Data: TanStack Query + Reactive Mock Engine	Form Validation: React Hook Form + Zod
Source of Truth: Stitch Luxecare UI (453 Screens)	Design Palette: Queryholic Dark Theme (Geist + Material Symbols)

1. Executive Summary & Screen Inventory Breakdown

A rigorous audit of the workspace revealed **453 screen directories** inside `stitch_luxecare_design_system`. The application covers a complete multi-sided marketplace: Customers, Service Providers (Vendors), Delivery Logistics Partners, and Platform Admin/Operations.

Role / Domain	Screens	Key Functional Scope
Customer Marketplace	237	Discovery, Categories, Provider Compare, Multi-Step Booking, Cart, Checkout, Order Tracking, History, Reviews, Profile, Addresses, Payments, Support
Admin & Operations	76	Live KPI Operations, Order Lifecycle Audit, Vendor/Driver KYC Verification, Dispute Investigation Room, Refunds, Settlements, Analytics, Audit Logs
Delivery Logistics	68	Driver Onboarding, Vehicle Docs, Job Pool, Pickup Workflow, OTP Verification, Active Turn-by-Turn Navigation, Delivery Proof Capture, Reschedule Flow, Earnings
Provider (Vendor)	54	Onboarding & KYC, Catalog Management, Dynamic Pricing & Variants, Capacity/Availability, Workshop Stages (Intake, Wash, Dry, Iron, QC, Handoff), Revenue & Payouts
Marketing & Platform	7	Landing Hero, Service Exploration, How It Works, B2B Commercial Laundry, Partner Acquisition (Driver/Vendor), Trust & Safety
Authentication	5	Multi-Role Login, Account Registration, Phone OTP Verification, Forgot Password, Security Sessions
System & Edge States	6	No Internet Offline, System Maintenance, Force App Update, Skeletons, Design System Reference
TOTAL	453	Complete Multi-Vendor Ecosystem (No screens omitted or simplified)

2. Design System & Visual Tokens (Locked)

The Stitch-generated UI is the visual source of truth. Default shadcn styling will be customized to reproduce the established Queryholic dark aesthetic:

Token Category	Hex / Value	Application / Usage
Background & Surface	#131314	Main app canvas, page backdrops, primary card base
Elevated Surface	#2a2a2b	Floating cards, dropdown menus, modals, dialog sheets
Surface Container	#201f20 / #353436	Inner card containers, input fields, search boxes
Primary Brand	#dcb8ff	Light Violet / Lilac for CTA buttons, active tabs, highlights
Primary Container	#8a2be2	Deep Purple accent for active icons, badge fills, banners
On-Primary Text	#480081	High contrast dark purple text on lilac primary buttons

Secondary / Muted	#c8c6c8 / #474649	Secondary buttons, inactive badges, border outlines
Status (Error/Alert)	#ffb4ab / #93000a	Validation errors, cancellation chips, destructive actions
Typography & Icons	Geist + Material Symbols	Clean modern grotesque font with Google Material Symbols Outlined

3. Multi-Tier Architecture & Backend-Ready Contract

To ensure clean separation of concerns and zero rewrite when integrating the real backend API, the frontend strictly follows the 4-tier layer pattern:

Layer	Responsibility & Implementation
1. Presentation (UI)	Next.js Server & Client Components, customized shadcn primitives, Tailwind layouts, responsive viewports.
2. Feature Hooks	TanStack Query custom hooks (useBookings, useProviderOrders, useDeliveryJobs, useKYC).
3. Service Layer	Typed service interfaces (customerService, providerService, deliveryService, adminService). When backend is ready, only this layer changes.
4. Mock Data Engine	Reactive in-memory + LocalStorage store with interconnected seed data reflecting the full lifecycle across all roles.

4. Step-by-Step Implementation Roadmap (Stages 1 to 8)

Stage 1: Core Foundation & Tokens

- Initialize Next.js App Router with TypeScript strict mode enabled.
- Install TanStack Query, Lucide React, class-variance-authority, React Hook Form, Zod.
- Configure Tailwind CSS with exact Stitch hex tokens, radius, and Geist font.
- Link Google Material Symbols Outlined font in root layout.

Stage 2: Reusable Shadcn Primitives & Shared UI

- Build customized primitives: Button, Input, Select, Badge, Card, Dialog, Sheet, Tabs, Table, Skeleton.
- Build marketplace components: ServiceCard, ProviderCard, ItemQuantityStepper, PriceBreakdownCard.
- Build logistics components: OrderStatusBadge, TimelineStepper, MetricStatCard, OtpInput, PhotoUploadDropzone.

Stage 3: Domain Types & Reactive Mock Store

- Define strict TypeScript entities: User, Customer, Provider, Driver, Service, Booking, Order, Payment, KYC, Dispute.
- Build mockStore.ts with realistic seed data and localStorage persistence.
- Implement lifecycle status updates (Customer Booking -> Provider Accept -> Driver Pickup -> Delivery -> Admin Settlement).

Stage 4: Role Layouts & Navigation Cockpits

- Public Marketing Layout: Header, sticky service links, vendor CTA, footer.
- Customer Layout: Location switcher, live cart badge, mobile bottom navigation.
- Provider Layout: Multi-section operational sidebar, online/offline switch, quick KPI bar.
- Delivery Layout: Mobile-first cockpit, turn-by-turn map overlay, floating job drawer.
- Admin Layout: High-density operational sidebar, global alert action center.

Stage 5: Core Feature Modules Implementation

- Customer Flow (237 screens): Discovery, 5-step booking, checkout, live tracking, order history, reviews.
- Provider Workshop (54 screens): Intake inspection, washing/drying queue, quality check, driver handoff, payouts.
- Delivery Logistics (68 screens): Job pool, OTP pickup, turn-by-turn navigation, photo proof, reschedule modal.
- Admin Control Center (76 screens): Master orders, KYC approvals, dispute investigation room, refund trigger.

Stage 6: Form Validation & Async States

- Implement Zod schemas + React Hook Form for Login, Registration, Addresses, KYC, Service Creation, Disputes.
- Add explicit Loading (skeletons), Empty (call-to-actions), Error (retry buttons), and Success states on all screens.

Stage 7: End-to-End Interactivity & Cross-Role Sync

- Connect full lifecycle: Order placed in Customer App immediately appears in Provider & Admin queues.
- Provider QC completion immediately triggers Available Delivery Job for drivers.
- Driver proof upload marks order Delivered and credits Provider/Driver earnings.

Stage 8: QA, TypeScript Check & Production Build

- Execute `npx tsc --noEmit` to guarantee 0 TypeScript errors.
- Execute `npm run build` to validate all App Router routes and static generation.
- Perform side-by-side visual QA against original Stitch screen references.

5. Role-by-Role Workflow & Interaction Mapping

A. Customer Journey

- Discovery & Catalog:** Browse categories (Wash & Fold, Dry Cleaning, Shoe Care, Bedding), filter by turnaround time, ratings, express tags.
- Customization:** Configure garment variants (e.g. Silk Shirt vs Wool Coat), select addon treatments (Stain Removal, Eco Detergent).
- Booking & Scheduling:** Pick pickup and return time windows, select saved address or add new with GPS pin.
- Checkout & Pay:** Apply coupon codes, pay via mock Card/UPI/COD, receive instant order confirmation.
- Live Tracking:** Animated status timeline, live driver location, driver contact, OTP authorization.

B. Provider Workshop Operations

- Onboarding & KYC:** Business details, GSTIN, store photos, trade license verification.
- Catalog & Pricing:** Create services, configure variant tiers, addon pricing, turnaround guarantees.
- Capacity & Working Hours:** Daily maximum order limits, holiday blackout dates, operating hours.
- Workshop Pipeline:** Item Intake Inspection (pre-wash stain photo capture) → Washing → Drying → Ironing → Quality Check → Packaged & Ready.
- Finance:** Real-time revenue dashboard, withdrawal requests, settlement audit logs.

C. Delivery Partner Logistics

- Driver Cockpit:** Shift Online/Offline switch, daily trip count, earnings goal progress.
- Job Acceptance:** Interactive job pool with distance, bag count, payout preview, accept/reject timer.
- Pickup Workflow:** Navigation to vendor/customer → Arrived → Customer OTP confirmation → Bag verification.
- Delivery & Proof:** Arrived at customer → Camera photo proof of delivery / signature → Mark Delivered.
- Issue Handling:** Customer unavailable, wrong address, damaged package rescheduling workflow.

D. Admin & Operations Command Center

- Operational Dashboard:** Platform order volume, live driver fleet map, active revenue.
- Master Orders Audit:** Comprehensive order table with deep timeline, manual stage override, invoice view.
- KYC Review Queue:** Inspect uploaded trade licenses, driving licenses, approve/reject with feedback.
- Dispute Investigation Room:** Dual-sided evidence viewer (customer photo vs provider intake photo), refund disbursement.
- Financial Reconciliation:** Platform commission tracking, vendor payout batching, gateway logs.

6. Quality Assurance & Verification Criteria

QA Dimension	Success Benchmark / Acceptance Criteria
Visual Fidelity	100% match with Stitch design system tokens (colors, font sizes, border radii, dark mode elevation).
TypeScript Strictness	Zero any types across all components; npx tsc --noEmit passes with 0 errors.
Production Build	npm run build generates all static and dynamic App Router routes successfully without warning.
Form Validation	All inputs have Zod schema validation, accessible error labels, and disabled states during submit.
Cross-Role State Sync	Customer orders seamlessly flow to Provider workshop, Driver job board, and Admin logs in mock store.
Responsive Design	Tested and optimized across Mobile (375px), Tablet (768px), and Desktop (1440px) viewports.